Agentic AI 7: Computer-use, coding and long-horizon agents

Course syllabus · version 1 · approved 2026-10-11

Who it's for: Developers who have done tracks 1–6 (or can write a tool-calling agent loop, use MCP, and know evals, tracing and guardrails). Comfortable with Python, git, pytest and basic Docker.

What you'll be able to do: Build browser and computer-use agents (aria snapshots and screenshots, Playwright, vendor computer-use tools) and coding agents (search/edit/test tools, sandboxes, CodeAct, MCP via code) that work unattended across many steps: manage context with notes, just-in-time retrieval, compaction, skills and subagents; survive failure with checkpoints, resumable runs, budgets and honest verifiers; evaluate them with SWE-bench-style harnesses; defend against prompt injection with sandboxes, credential brokering and design patterns; and reason about fleets of agents and the research behind them.

Running example — Pocket Tracker, a tiny issue tracker with its own repo: Everyone understands issues and a tracker. One local artefact (a FastAPI + SQLite app with a pytest suite and seeded bugs filed in issues.json) serves both agent types: browser/QA agents drive the running app, coding agents fix its repo. Fully offline and deterministic; it grows into feature backlogs, parallel fleets and many tenants.

How it grows: simulated tracker dict -> real app driven by Playwright -> browser agent that reproduces bugs -> coding agent that fixes one issue in a sandbox -> QA agent that files bugs via MCP -> five issues in one session -> 50+ step feature build -> production fixer with evals and injection defences -> fleet of agents on the backlog -> capstone QA engineer and fixer

79 lessons · about 54 hours · 9 parts

Choose your path

Nothing is left out: Complete is the whole course. Shorter paths take the most-used lessons first; each works on its own.

Path	Lessons	Time	Includes
Essentials	43	~25 h	Core lessons: what most real work uses
Professional	68	~46 h	Core + Professional: build, ship and run it in production
Complete	79	~54 h	Everything, including internals, enterprise scale and the frontier

Or as three courses by difficulty

Course	Levels	Lessons	Time
Basic	L1 Foundations + L2 Core	00–21 (24)	~14 h
Intermediate	L3 Practical + L4 Advanced	22–45 (30)	~20 h
Advanced	L5 Production + L6 Enterprise	46–66 (25)	~20 h

LLM use: LLM-centric and LLM-based track (user decision 2026-10-11, overriding LEVELS rule 10's fake-model-first for this topic): a real model drives every lesson where an agent acts, recorded with llm_replay so it runs without a key. A scripted fake model appears only in lesson 46a (testing agents with a fake is the skill) and injected faults only in chaos drills. Use real, standard libraries, not course-made helpers: models from LangChain init_chat_model with bind_tools / AIMessage.tool_calls / ToolMessage; Playwright, the Docker SDK, mcp, pytest and git for everything else. Each lesson writes its own small agent loop visibly in the notebook; no hidden helper modules. The only course helper is llm_setup.get_llm(<lesson>), which lists every provider and model with key status and recordings, starts llm_replay with one recording per model, and returns the LangChain model; learners switch model in .env or with get_llm(..., provider=, model=). A model with no recording and no key stops with a clear error naming the key to add. Lessons that teach one vendor's own format (13, 13a, 32a) call that SDK directly. Prose never depends on one model's exact output. Recordings: Gemini while building (GOOGLE_CLOUD_API_KEY), plus Claude/OpenAI when those keys are present; never invented. Low cost (user decision 2026-10-11): every example makes at most 2 model calls (e.g. actor-critic = 1 actor + 1 critic call); a lesson's real-call budget is its llm_calls value (default 4), enforced by llm_replay, which refuses any call over budget and prints the count. Long-run lessons are sized down (a few tasks, 1 seed, 2 agents) with llm_calls of at most 20, using the cheapest model; batch tool calls per turn. Replay makes no calls. Model calls are recorded once and replayed, so learners

need no API key.

Tools used

Tool	Role	Lessons
LangChain init_chat_model (Gemini, Claude, OpenAI, Ollama) + llm_replay via llm_setup.get_llm	the real model in every agent lesson; learner picks any provider, key status shown; one recording per model	01, 05, 09, 12 onward
Playwright 1.63 (headless Chromium)	browser control, aria snapshots, screenshots, traces	07-14, 27-28, 42, 47, 49, 54, 63, 66
FastAPI + SQLite + pytest (Pocket Tracker in apps/)	the environment and the repo the agents work on	06-66
Docker (python:3.12-slim, hardened flags)	coding-agent sandbox, parallel agents, desktop sandbox	19, 21, 36, 38, 44-46, 52-53, 55-57, 60, 64, 66
anthropic, openai SDKs	computer-use tool formats and compaction/context-editing features; real calls recorded only if a key was present while building	13, 32
mcp (Python SDK 2.x)	local Pocket Tracker MCP server for MCP-via-code and bug filing	25, 28, 66
mini-swe-agent, browser-use, Stagehand	reference agents compared with your own	23, 27
Claude Code, Codex CLI, Gemini CLI / Antigravity, Claude Agent SDK, OpenHands SDK	real coding agents shown through recorded transcripts and headless runs	15, 20, 26, 33, 35, 58

Lessons

Priority: **Core** = Essentials path · **Pro** = Professional path · **Deep** = Complete path. ★ = checkpoint project.

Part 1 · L1 Foundations: What makes these agents different

#		Lesson	You will be able to	Needs	Min
00	Core	**Introduction: computer-use, coding and long-horizon agents**	Explain what computer-use, coding and long-horizon agents are, where they are used at work, and what you will build in this course; set up your model and see which keys you have.	llm	20
01	Core	**Agents that act on a computer**	See, with a real model, how a computer-use or coding agent differs from a chat agent: it observes an environment, acts on it through tools, and must check the result.	llm	20
02	Core	**Observations: pixels, accessibility trees and DOM**	Compare the three ways an agent can see a page and estimate each one's token cost.	—	25
03	Core	**Actions: coordinates, element refs, keys, shell and code**	Name the main action spaces (pixel coordinates, element references, keyboard, shell, code) and say which observation each needs.	—	25
04	Core	**Why long tasks fail: compounding errors and time horizons**	Show with a simulation why per-step reliability matters so much over 50+ steps, and read a METR-style time-horizon chart, including why horizons above ~16 hours are not yet reliable.	—	25
05	Core	**Checkpoint: a real-model agent in a simulated tracker ★**	Build a complete observe-decide-act-verify loop against a simulated environment, with a step limit and an outcome check.	llm	60

Part 2 · L2 Core: Driving a browser

#		Lesson	You will be able to	Needs	Min
06	Core	**Meet Pocket Tracker: the app and its repo**	Start the Pocket Tracker web app and its git repo locally and find your way around both.	—	25
07	Core	**Playwright: launching headless Chromium**	Open, navigate and read a page with Playwright's sync API in headless Chromium.	download	30
08	Core	**Aria snapshots as observations**	Use page.aria_snapshot() (including element refs) as a compact, text-only observation for a model.	download	30
09	Core	**Acting by element reference**	Turn a model's chosen element ref into a Playwright click, fill or select, and handle a stale ref.	download, llm	30
10	Core	**Screenshots: size, scaling and token cost**	Take screenshots at a chosen viewport, compute their image-token cost and keep cassettes small.	download	30
11	Pro	**Grounding: coordinate systems and off-by-N clicks**	Convert model coordinates (scaled pixels, 0-999 grids) to page pixels and diagnose the scaling bug.	download	30
12	Core	**The browser agent loop with a real model**	Run an aria-snapshot browser agent with a real model under llm_replay, with a step limit and an outcome check.	download, llm	35
13	Pro	**Vendor computer-use tools: the Claude computer toolset**	Read and handle Claude's computer_toolset_20260801 messages (one tool per action, no beta header), including zoom and the resolution table.	download, llm	25
13a	Pro	**Vendor computer-use tools: OpenAI and Gemini**	Handle OpenAI's computer tool (computer_call actions, plus GPT-6 Astra's exec_py code-driven style) and Gemini's computer_use tool (normalized coordinates, safety_decision), and compare them with Claude's.	download, llm	30
14	Core	**Checkpoint: a browser agent that reproduces a bug ★**	Build a browser agent that follows a bug report, reproduces the bug in the app and saves evidence (screenshots and aria snapshots).	download, llm	90

Part 3 · L2 Core: Coding-agent primitives

#		Lesson	You will be able to	Needs	Min
15	Core	**Inside a coding agent: gather context, act, verify**	Describe the loop and tool categories of Claude Code, Codex and Gemini CLI / Antigravity from real transcripts.	llm	30
16	Core	**Repo search tools: glob, grep and windowed viewing**	Write search and file-view tools whose output fits a model's context.	llm	30
17	Core	**Edit tools: exact-string replace, patches and rewrites**	Implement an exact-string edit tool and explain why it beats whole-file rewrites; handle no-match and multi-match errors.	llm	30
18	Core	**Running tests as the verifier**	Run tests from a tool, keep the output short, and use a failing reproduction test before fixing.	llm	30
19	Core	**A Docker sandbox for agent commands**	Run an agent's shell commands in a hardened, network-less container with resource limits and timeouts.	docker	35
19a	Pro	**Building the sandbox: per-repo images and offline dependencies**	Build a per-repo sandbox image with an init.sh, install dependencies at build time, and run the agent with no network or a package-index allowlist.	docker	30
20	Core	**Instruction files: AGENTS.md, CLAUDE.md, GEMINI.md**	Write a short, high-signal instruction file and explain how nesting and precedence work.	llm	25

#		Lesson	You will be able to	Needs	Min
21	Core	**Checkpoint: fix one issue in a sandbox ★**	Build a small coding agent that reads an issue, finds the code, edits it, runs the tests in a sandbox and stops when they pass.	docker, llm	90

Part 4 · L3 Practical: Better tools and action spaces

#		Lesson	You will be able to	Needs	Min
22	Pro	**Agent-computer interfaces: tools designed for models**	Apply the SWE-agent interface ideas (simple actions, compact feedback, lint-on-edit guardrails) and count their effect with a minimal 3-issue pass/fail scorer.	llm	35
23	Pro	**Bash-only agents: mini-swe-agent**	Run mini-swe-agent on Pocket Tracker and explain why a bash-only, linear-history agent scores close to complex scaffolds.	llm	30
24	Pro	**Code as the action space: CodeAct**	Let the model act by writing Python in a sandboxed interpreter, and compare it with JSON tool calls.	llm	35
25	Pro	**Calling MCP tools by writing code**	Expose MCP tools (stateless MCP 2026-07-28, no initialize handshake) as a code API the agent discovers on disk, keep intermediate data in the sandbox, and measure the token saving; contrast with MCP tool search for tool overload.	llm	35
26	Core	**Driving coding agents from code: headless mode and SDKs**	Run coding agents non-interactively (claude -p, codex exec, Claude Agent SDK, Codex SDK / App Server, OpenHands SDK) and parse their output.	cloud	30
26a	Core	**Coding agents in CI and pull requests**	Run an agent in a CI job that opens a pull request, and review an agent's diff the way a human reviewer should (scope, tests touched, secrets, flaky checks).	llm	30
27	Pro	**Browser agent libraries: browser-use and Stagehand**	Use browser-use and Stagehand against the local app, and say what they add over your own loop.	download, llm	30
27a	Core	**Playwright MCP: the browser coding agents use**	Give an agent a browser through the Playwright MCP server (aria-snapshot tools) and compare it with the hand-built loop from lesson 12.	download, llm	30
28	Pro	**Checkpoint: a QA agent that writes browser tests and files bugs ★**	Build an agent that reads a feature spec, writes and runs Playwright tests, and files failures through the tracker's MCP server.	download, llm	120

Part 5 · L3 Practical: Context for long tasks

#		Lesson	You will be able to	Needs	Min
29	Core	**Context rot: recall falls as the window fills**	Measure how a model's recall of an early fact drops as tool output piles up.	llm	30
30	Core	**Progress notes and external memory files**	Give the agent a notes file it writes and re-reads, so it keeps state outside the context window; compare with vendor memory (memory_20250818, Claude Code auto memory).	llm	30
31	Core	**Just-in-time retrieval: paths, not blobs**	Keep references (paths, queries) in context and load content only when needed.	llm	30
32	Core	**Compaction by hand**	Compact a long history by hand and see what the agent forgets.	llm	25
32a	Pro	**Vendor context management: compaction, tool-result clearing and the cache**	Use the vendor features (compact_20260112, clear_tool_uses_20250919) and measure how compaction or a model switch invalidates the prompt cache.	llm	30

#		Lesson	You will be able to	Needs	Min
33	Core	**Agent Skills: expertise loaded on demand**	Write and validate an Agent Skill (SKILL.md, scripts, references), load it progressively in your own agent, and test that it triggers when it should and not otherwise.	llm	35
34	Core	**Subagents for a clean context window**	Delegate exploration to a subagent that returns a short summary, and measure what the parent saves.	llm	30
35	Core	**Permission modes and allow/deny rules**	Implement permission modes (Manual, Accept edits, Plan, Auto) with allow/deny rules, map them to Codex's sandbox_mode x approval_policy, and explain why a classifier is not an isolation boundary.	llm	25
35a	Core	**Hooks: deterministic guardrails around tools**	Add pre-tool and post-tool hooks that block or fix actions deterministically, whatever the model decides.	llm	25
36	Core	**Checkpoint: three issues in one session ★**	Extend the fixer to work through several issues in one session without running out of context, using notes, JIT retrieval, a skill and guardrails.	docker, llm	120

Part 6 · L4 Advanced: Reliability on one machine

#		Lesson	You will be able to	Needs	Min
37	Core	**The long-run harness: initializer, feature list and progress file**	Build the initializer-plus-worker harness: a JSON feature list, a progress file, a commit per feature and a smoke test at the start of every session.	llm	35
38	Core	**Checkpoints, rewind and resumable runs**	Kill a run mid-task and resume from git plus the progress file (session --resume / --fork-session), retry provider 429/overload errors with backoff, and rewind a bad edit from a checkpoint.	docker, llm	35
39	Core	**Knowing when to stop: done checks, loop detection and budgets**	Stop runs that loop, stall or overspend (step, token and wall-time budgets), and stop agents declaring success early.	llm	30
40	Core	**The verifier problem: reward hacking in coding agents**	Recognise agents gaming tests (editing tests, special-casing inputs) and flaky tests that make verifiers lie, and build checks that catch both.	llm	30
41	Pro	**Agentless vs agent loop: when a fixed pipeline wins**	Build a localize-repair-validate pipeline and compare it with the agent on pass rate and cost, using the 5-issue scorer from lesson 22.	llm	35
42	Pro	**Why real sites break browser agents**	Diagnose the common breakages (cookie banners, popups, iframes, native dialogs, late-loading content, anti-bot) and add handling for each.	download, llm	35
42a	Pro	**Logged-in browsing: storage state, profiles and handing over to a human**	Reuse login state safely (storage state, persistent profiles) and hand 2FA or CAPTCHA steps to a human instead of solving them.	download	25
43	Core	**The cost of long runs: tokens per step, images and caching**	Turn step and token budgets into dollars: a cost model for a 50-step run, and the levers (text observations, caching, smaller models per role, pruning images).	llm	30
44	Core	**Parallel agents: worktrees and one container each**	Run several agents at once with a git worktree and a container per agent, and see what worktrees do not isolate.	docker, llm	35
44a	Core	**Structured run logs and transcripts**	Log every step (observation, action, tokens, cost) as a JSONL transcript, redact secrets from it, and find a failure from it.	llm	25

#		Lesson	You will be able to	Needs	Min
45	Pro	**Checkpoint: a long unattended run, every failure explained ★**	Run the long-run harness unattended (up to 20 model calls) and write a failure analysis from its logs.	docker, llm, slow	120

Part 7 · L5 Production: Shipping safely

#		Lesson	You will be able to	Needs	Min
46	Core	**An eval harness: fail-to-pass and pass-to-pass**	Build a SWE-bench-style harness: instances with problem statement, patch, hidden tests, and a JSON report.	docker, llm	35
46a	Pro	**Testing the agent itself**	Unit-test an agent with a scripted model, guard it with cassette regression tests, and use fixture sites for browser agents.	—	25
47	Pro	**Evaluating browser agents: outcome graders and variance**	Grade browser agents on end state and run multiple seeds to report pass rate with a confidence interval.	download, llm	30
47a	Pro	**LLM judges for agent runs: checking agreement**	Check an LLM judge against hand labels before trusting it for runs that have no state to grade.	llm	25
48	Pro	**Reading benchmarks: SWE-bench, OSWorld, WebArena, Terminal-Bench**	Read a benchmark claim critically: task source, grader, contamination, harness versus model, cost, and which variant is current (SWE-bench Pro, Terminal-Bench 4.0, OSWorld 2.0, WebArena-Verified; why SWE-bench Verified and original WebArena are no longer the headline).	—	30
49	Core	**Prompt injection from web pages and repos**	Show an injection through a web page and through a repo file, and measure attack success on a tiny local suite.	download, llm	35
50	Pro	**Defences by design: plan-then-execute, dual LLM and CaMeL**	Apply design patterns where untrusted text cannot choose actions, and re-measure attack success.	llm	35
51	Core	**Human confirmation for consequential actions**	Gate consequential actions with the real mechanisms (Gemini safety_decision.require_confirmation, OpenAI safety checks, Claude Code ask rules) and your own confirmation step.	llm	25
52	Pro	**The sandboxing ladder: hardened containers, gVisor and microVMs**	Climb the sandboxing ladder (hardened containers, gVisor runsc, microVMs): run the same agent under runsc and measure start time and overhead.	docker	35
53	Pro	**Secrets stay outside: credential brokering and egress allowlists**	Keep tokens out of the sandbox with a credential proxy and restrict outbound traffic to an allowlist.	docker	30
54	Core	**Observability: OpenTelemetry spans and Playwright trace files**	Export agent runs as OpenTelemetry spans and Playwright trace files, and find a failure from them.	download	30
55	Deep	**Desktop computer use: a virtual display in Docker**	Run a desktop sandbox (Xvfb, window manager, VNC) in plain Docker and drive a GUI app with screenshot and mouse/keyboard actions.	docker, download, llm	35
56	Pro	**Checkpoint: a production-ready Pocket Tracker fixer ★**	Ship the fixer with evals, injection defences, approvals, a hardened sandbox, brokered credentials, budgets and tracing.	docker, llm	120

Part 8 · L6 Enterprise: Fleets and organisation scale

#		Lesson	You will be able to	Needs	Min
57	Deep	**Fleets: planners, workers, judges and task claiming**	Coordinate a fleet (planners, workers, judges, task claiming via lock files) and measure lock contention: why 20 agents under one shared lock behave like 2-3.	docker, llm	35
58	Pro	**Coding agents across an organisation: managed policy and shared skills**	Enforce organisation-wide policy (Claude managed settings and sandbox keys via MDM, ConfigChange audit hooks, Codex sandbox_mode x approval_policy x approvals_reviewer) and share skills across teams.	—	30
58a	Deep	**Rolling out new agent, model and tool versions without downtime**	Roll out a new model or computer-use tool version with rainbow deployment, run old and new side by side, and migrate long-running sessions safely.	docker, llm	30
59	Deep	**Case studies: a C compiler, a browser in a week, and harness engineering**	Use published write-ups (Anthropic, Cursor, OpenAI) to justify design choices with their numbers.	—	30
59a	Deep	**Failure drill: a fleet under chaos**	Kill sandboxes, crash Chromium and inject 429s into a running fleet, then measure recovery time and duplicated tool side effects.	docker, slow, llm	35
60	Deep	**Checkpoint: three agents, one backlog ★**	Run a small fleet on Pocket Tracker's backlog with task claiming, isolation and a judge, and report throughput, cost and conflicts.	docker, llm, slow	120

Part 9 · L6 Enterprise: Research, frontier and capstone

#		Lesson	You will be able to	Needs	Min
61	Deep	**Reading agent papers: from ReAct to OSWorld 2.0**	Place the key papers on one map (what each changed) and read one critically.	—	30
62	Deep	**Running a fair agent experiment**	Design an experiment with seeds, variance, fixed harness and cost reporting so a result means something.	—	30
63	Deep	**Reproduction: aria snapshots vs screenshots**	Measure success, tokens and cost for text versus image observations on local tasks and compare with published claims.	download, llm	40
64	Deep	**Checkpoint: reproduce the agent-computer interface ablation**	Reproduce the SWE-agent interface finding (now historical; mini-swe-agent is its successor) at tiny scale and write up what did and didn't match.	docker, llm, slow	90
65	Deep	**The frontier: the last year in computer-use and coding agents**	Summarise the most important changes of the last 12 months, with dates and sources, and where the field is going.	—	30
66	Pro	**Capstone: an autonomous QA engineer and fixer for Pocket Tracker ★**	Combine the QA agent and the production fixer: read a spec, test in the browser, file bugs via MCP, fix them in a sandbox, unattended, with evals and a failure analysis.	docker, download, llm, slow	240

Enterprise depth coverage

Checklist item	Lessons
1 distributed architecture internals	N/A — covered in track agentic-ai-6-production (runtime architecture and durable execution: agentic-ai-5-multi-agent); fleet coordination designs previewed in lesson 57
2 cluster operations hands-on	N/A — covered in track agentic-ai-6-production (scaling agent workers and queues); parallel agent containers in lessons 44, 60
3 failure and recovery	59a
4 extreme scale	N/A — covered in track agentic-ai-6-production (millions of runs per day); fleet numbers from case studies in lesson 59
5 kubernetes and infrastructure	N/A — covered in track agentic-ai-6-production (deploying agent runtimes and sandboxes on Kubernetes)
6 performance engineering at scale	N/A — covered in track agentic-ai-6-production (load/latency/cost testing); per-run cost model in lesson 43
7 high availability and disaster recovery	N/A — covered in track agentic-ai-6-production (HA/DR for agent runtimes); single-run resume in lesson 38
8 zero-downtime change	N/A — covered in track agentic-ai-6-production (prompt/model/tool versioning and rollout); agent-specific rainbow deployments and computer-use tool-version migration in lesson 58a
9 multi-tenancy and isolation	N/A — covered in track agentic-ai-6-production (per-tenant agents, data and budgets); per-agent sandbox isolation in lessons 44, 52
10 security and compliance at org scale	N/A — covered in tracks agentic-ai-4-protocols and agentic-ai-6-production; agent-specific sandboxing, credential brokering, injection and policy in lessons 49-53, 58
11 cost engineering	N/A — covered in track agentic-ai-6-production (token cost engineering); long-run cost levers in lesson 43
12 platform and organisation patterns	N/A — covered in track agentic-ai-6-production (agent platforms and governance); coding-agent policy, shared skills and CI agents in lesson 58
13 frontier	65
14 case studies	59

Your syllabus → lessons

Source item	Lessons
Level 7 › Screenshot → action loops	10, 12, 13
Level 7 › Accessibility trees versus pixels	02, 08, 10, 63
Level 7 › Playwright-based browser agents	07, 09, 12, 27, 27a

Source item	Lessons
Level 7 › Benchmarks: OSWorld, WebArena, WebVoyager	48, 61
Level 7 › Why reliability on real sites is still hard	42, 48
Level 7 › How Claude Code, Codex and similar agents work: repo search, edit tools, running tests, sandboxes	15, 16, 17, 18, 19, 26, 19a, 26a
Level 7 › Agent-computer interfaces (the SWE-agent idea)	22, 23, 64
Level 7 › Code execution as the action space (CodeAct)	24
Level 7 › Agents that call MCP tools by writing code	25
Level 7 › Context engineering at scale: compaction	32, 32a
Level 7 › External memory files, progress notes	30, 37
Level 7 › Just-in-time retrieval	31
Level 7 › Planning that survives failure: checkpoints, resumable runs	37, 38
Level 7 › Self-verification with tests	18, 40
Level 7 › Skills: packaged instructions and scripts loaded only when needed	33
Level 7 › Training models for agency (SFT, RLVR, GRPO, gyms, small open models)	out of scope — track agentic-ai-8-training (see out_of_scope)
Level 7 › Build: coding agent fixing real issues from a small repo's tracker in a Docker sandbox	21, 36, 56
Level 7 › Build (alternative): RL fine-tune of a 7B model	out of scope — track agentic-ai-8-training
Level 7 › Exit test: 50+ step task unattended, explain each failure in its logs	45, 66
Capstone 3 › Autonomous QA engineer (spec → browser tests → bugs filed via MCP)	28, 66

Out of scope

Training models for agency: SFT on trajectories, RLVR/GRPO/PPO, agent gyms (SWE-Gym, SWE-smith, Agent-R1), fine-tuning small open models, GUI-native models (UI-TARS, Aguvis) — Track agentic-ai-8-training (needs GPU).

MCP protocol internals, OAuth for tools, tool poisoning, A2A, AG-UI — Track agentic-ai-4-protocols; this track only uses a local MCP server.

Durable-execution engines (Temporal, LangGraph checkpointers) and general multi-agent orchestration patterns — Track agentic-ai-5-multi-agent; this track uses git + progress files for resumability.

Gateways, OpenTelemetry pipelines, multi-tenant budgets, Kubernetes deployment of agent runtimes — Track agentic-ai-6-production.

Running full benchmark suites locally (SWE-bench needs ~120 GB disk, OSWorld VM images, WebArena sites) — Too heavy for a laptop; taught read-along in lesson 48 with self-built mini harnesses in 46, 47, 63, 64.

Automating real third-party websites — Not reproducible (sites change, CAPTCHAs, terms of service); taught with the local app's hostile mode and recorded failures in lesson 42.

Syllabus history

Version	Date	Changes
1	2026-10-11	first approval